Depo Robotu Simülatörü: Dosya Tabanlı Kural Motoru ve Tkinter Arayüzü

Nijat Mammadov
240501056

Asgaragha Zarbaliyev
240501048

Kocaeli Sağlık ve Teknoloji Üniversitesi, YZM217/BLM217 Programlama Lab-I
2026–2027 Güz Dönemi, Proje 1

***Özet*—Bu çalışmada, tek bir robotun bir depoda kutuları alıp teslim noktalarına taşıdığı ve bataryası azaldığında şarj olduğu bir simülatör Python 3 ile geliştirilmiştir. Program harita, komut ve ayar dosyalarını okur; komutları hareket, batarya, yük ve çarpışma kurallarına göre adım adım çalıştırır; sonuçları JSON ve CSV dosyalarına yazar ve robotu Tkinter arayüzünde gösterir. Kurallar ile arayüz ayrı modüllerde tutulmuş, hatalı komut satırları programı durdurmadan satır numarasıyla raporlanmıştır. Bonus olarak GIT x y komutu, genişlik öncelikli arama (BFS) ile duvarlardan kaçınan en kısa yolu bulacak şekilde gerçeklenmiştir. Verilen altı örnek testin tamamı geçilmiş, beş deneme veri setinde tüm kutular teslim edilmiş ve 3000 rastgele senaryoda program çökmemiştir.**

***Anahtar Kelimeler*—simülasyon, iki boyutlu liste, dosya işlemleri, Tkinter, genişlik öncelikli arama, hata yönetimi.**

I. GİRİŞ

Depo otomasyonunda robotların kutu taşıma görevleri, enerji kısıtları ve engeller altında planlanmalıdır [1]. Bu projede bu problemin basitleştirilmiş bir modeli ele alınmıştır: ızgara şeklinde bir depoda tek bir robot, bir metin dosyasındaki komutları sırayla uygular. Robotun her adımı ve dönüşü batarya harcar, kutu taşırken adım maliyeti artar, duvara çarpmak ceza getirir ve batarya yetmediğinde simülasyon durur.

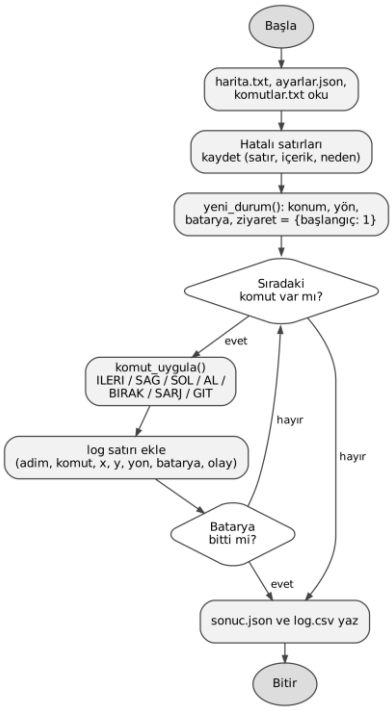
Projenin amacı; iki boyutlu listeler, TXT/JSON/CSV dosya okuma-yazma, metin ayrıştırma, fonksiyonlarla programlama, hata yönetimi ve arayüz geliştirme konularını uygulamalı olarak öğrenmektir. Program yalnızca Python standart kütüphanesi ile yazılmış; arayüz için Tkinter [2], dosya biçimleri için json ve csv modülleri [3], [4] kullanılmıştır. Değerlendirme, dersin sağladığı test_calistir.py betiğinin zorunlu fonksiyonları çağırıp sonucu beklenen değerlerle karşılaştırmasıyla yapılmaktadır.

Raporun II. bölümünde veri yapıları, kurallar ve algoritmalar; III. bölümünde test ve deneme verisi sonuçları; IV. bölümünde ise sonuçlar ve geliştirme önerileri sunulmuştur.

II. YÖNTEM

A. Genel Mimari

Program iki modülden oluşur. simulasyon.py dosya okuma, kurallar ve çıktı üretimini içerir ve tkinter içe aktarmaz; böylece test betiği bu modülü arayüz olmadan doğrudan çağırabilir. arayuz.py ise yalnızca gösterim ve kullanıcı etkileşiminden sorumludur ve kurallar için simulasyon.py fonksiyonlarını çağırır. Programın akışı Şekil 1'de verilmiştir.



Şekil 1. Programın akış diyagramı.

B. Veri Yapıları

Harita, izgara[y][x] biçiminde erişilen iki boyutlu bir karakter listesi olarak tutulur; x sütunu, y satırı gösterir ve sol üst köşe (0, 0)'dır. Robotun başlangıç hücresi ızgarada boş hücre ('.') olarak saklanır, robotun konumu ise ayrı tutulur. Böylece robot hareket ettiğinde ızgarayı güncellemek gerekmez.

Simülasyonun tüm anlık bilgisi tek bir durum sözlüğündedir: konum (x, y), yön, batarya, yük bilgisi, teslim ve çarpışma sayıları, toplam adım, batarya bitti bayrağı, ızgaranın bir kopyası ve hücre ziyaret sayılarını tutan bir sözlük. Yönler saat yönünde [K, D, G, B] listesinde sıralıdır; sağa dönüş indeksi 1 artırır, sola dönüş 1 azaltır ve sonuç mod 4 alınır. Her yönün (dx, dy) karşılığı ayrı bir sözlükte tutulur; örneğin K için (0, −1).

C. Dosya Okuma ve Hata Yönetimi

harita_oku sondaki boş satırları atar, her satırın uzunluğunu ilk satırla karşılaştırır ve tanınmayan karakterleri reddeder. Bulunan R hücreleri bir listede toplanır; liste boşsa veya birden fazla eleman içeriyorsa ValueError fırlatılır. ayarlari_oku eksik anahtarları varsayılan değerlerle tamamlar ve negatif ya da sayı olmayan değerleri reddeder.

komutlari_oku dosyayı satır satır işler. Boş satırlar atlanır ancak satır numarasına dahil edilir. Komut adları büyük/küçük harfe duyarlıdır; ILERI için tam olarak bir pozitif tam sayı beklenir (ILERI 0, ILERI −1, ILERI 2.5 ve parametresiz ILERI

hatalıdır), diğer komutlar parametre almaz. Hatalı satırlar {satir, icerik, neden} sözlükleri olarak listelenir. Arayüzde dosya bulunamaması ve bozuk harita gibi durumlar try/except ile yakalanır ve kullanıcıya mesaj kutusuyla bildirilir. Ele alınan hata durumları ve programın tepkisi Tablo I'de özetlenmiştir.

TABLO I
HATA DURUMLARI VE PROGRAMIN TEPKİSİ

Durum	Tepki
Dosya bulunamadı	FileNotFoundError yakalanır, mesaj
Haritada R yok / birden fazla R	ValueError, mesaj
Satır uzunlukları farklı	ValueError (satır no ile)
Haritada tanınmayan karakter	ValueError (x, y ile)
ayarlar.json bozuk / negatif değer	ValueError, mesaj
Bilinmeyen komut (ILER, GERI, sol)	Satır atlanır, kaydedilir
Eksik/fazla/geçersiz sayı	Satır atlanır, kaydedilir
Boş satır	Atlanır, numara sayılır

D. Hareket ve Batarya Kuralları

ILERI n komutu tek seferde değil, adım adım işlenir (Algoritma 1). Her adımda önce robotun önündeki hücre kontrol edilir: duvar veya harita dışıysa çarpışma sayısı artar, batarya çarpışma cezası kadar azalır ve komutun kalanı iptal edilir. Ceza bataryayı eksiye düşürecekse batarya 0 yapılır ve simülasyon durur. Hücre geçilebilirse adım maliyeti hesaplanır; robot kutu taşıyorsa maliyet yük çarpanı ile çarpılır. Maliyet bataryadan büyükse işlem yapılmaz ve simülasyon durur; eşitse işlem yapılır ve batarya 0 olur. Dönüşler de aynı _harca yardımcı fonksiyonu ile ücretlendirilir.

Algoritma 1: ILERI n komutu

```
for i ← 1 to n:
  (nx, ny) ← (x, y) + YON_VEKTORU[yon]
  if (nx, ny) duvar veya harita dışı:
    carpisma ← carpisma + 1
    if ceza > batarya:
      batarya ← 0; bitti ← true
      return BATARYA_BITTI
    batarya ← batarya − ceza
    return CARPISMA
  m ← adim_maliyeti × (yuk ? yuk_carpani : 1)
  if m > batarya:
    bitti ← true; return BATARYA_BITTI
  batarya ← batarya − m; (x, y) ← (nx, ny)
  ziyaret[(x, y)] ← ziyaret[(x, y)] + 1
return HAREKET
```

E. Yük, Şarj ve Ziyaret

AL yalnızca robot bir kutu hücresindeyse ve yük taşımıyorsa çalışır; kutu ızgaradan silinir. BIRAK teslim noktasında teslim sayısını artırır, boş hücrede ise kutuyu o hücreye bırakır. SARJ yalnızca şarj istasyonunda bataryayı kapasiteye doldurur. Yanlış yerde yapılan bu üç komut hiçbir şeyi değiştirmez ve olay GECERSIZ olarak kaydedilir. Ziyaret sözlüğü başlangıç hücresi için 1 ile başlar ve robotun girdiği her hücrede artırılır. En çok ziyaret edilen hücre, (−sayı, y, x) anahtarına göre en küçük eleman seçilerek bulunur; böylece eşitlikte önce y'si, sonra x'i küçük olan hücre seçilir. Her komutun sonucu, log.csv'ye yazılan bir olay adıyla temsil edilir (Tablo II).

TABLO II
KOMUT OLAYLARI

Olay	Anlamı
HAREKET	ILERI/GIT tüm adımları tamamladı
CARPISMA	Duvar/harita dışı; komutun kalanı iptal
DONUS	SAG veya SOL uygulandı
AL	Kutu alındı
TESLIM	Kutu T üzerinde teslim edildi
BIRAKILDI	Kutu boş hücreye bırakıldı
SARJ	Batarya kapasiteye dolduruldu
GECERSIZ	AL/BIRAK/SARJ yanlış yerde
BATARYA_BITTI	Maliyet karşılanamadı, simülasyon durdu
YOL_YOK	GIT hedefine yol bulunamadı

F. Bonus: GIT x y

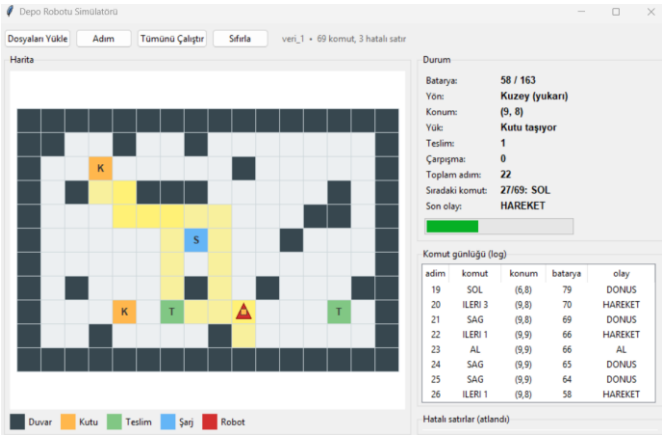
GIT komutu, robotun bulunduğu hücreden hedefe duvarlardan kaçınan en kısa yolu genişlik öncelikli arama ile bulur (Algoritma 2) [5], [6]. BFS hücreleri başlangıca olan uzaklık sırasıyla gezdiği için hedefe ilk ulaşıldığında bulunan yol, adım sayısı bakımından en kısadır. Her hücrenin hangi hücreden gelindiği onceki sözlüğünde tutulur; bu sözlük aynı zamanda ziyaret edilmiş hücreleri işaretleyerek sonsuz döngüyü önler. Yol bulunduktan sonra robot her adımda gereken yöne döner (180° için iki sağ dönüş) ve normal adım kuralları uygulanır; bu nedenle dönüş, adım ve batarya maliyetleri GIT için de geçerlidir. Hedef duvar, harita dışı ya da ulaşılamazsa olay YOL_YOK olur.

Algoritma 2: BFS ile en kısa yol

```
onceki ← {bas: YOK}; Q ← [bas]
while Q boş değil:
  h ← Q.popleft()
  if h = hedef:
    yolu onceki ile geri izle, ters çevir
    return yol
  for k in komşular(h) (K, D, G, B):
    if k geçilebilir and k ∉ onceki:
      onceki[k] ← h; Q.append(k)
return YOK
```

G. Arayüz ve Çıktılar

Arayüz (Şekil 2) Dosyaları Yükle, Adım, Tümünü Çalıştır ve Sıfırla butonlarından oluşur. Harita bir Canvas üzerinde renkli hücrelerle çizilir; robot baktığı yönü gösteren bir üçgenle, taşınan kutu robotun üzerinde küçük bir kareyle gösterilir. Robotun geçtiği hücreler ziyaret sayısına göre koyulaşan sarı tonlarla boyanır. Sağdaki bilgi panelinde batarya, yön, konum, yük, teslim, çarpışma ve sıradaki komut; altında ise renkli komut günlüğü ve atlanan hatalı satırlar yer alır. Adım butonu her basışta tek bir komutu komut_uygula ile çalıştırır. Simülasyon bittiğinde sonuc.json ve log.csv yüklenen klasöre yazılır; log.csv her komut için adim, komut, x, y, yon, batarya ve olay sütunlarını içerir.



Şekil 2. Arayüzün veri_1 üzerinde 26. komuttan sonraki görünümü.

III. Deneysel Sonuçlar

A. Örnek Test Paketi

Dersin sağladığı test_calistir.py betiği paket_ornek klasöründeki altı testle çalıştırılmıştır. Testler düz hareket, duvara çarpma, kutu alıp teslim etme, şarj istasyonu, hatalı komut satırları ve R içermeyen harita durumlarını kapsar. Tablo IV'te görüldüğü gibi tüm testler geçilmiştir (6/6). Ayrıca proje tanımındaki #R.K.T# örneği çalıştırılmış; konum ve batarya değerleri her adımda tanımdaki tabloyla birebir aynı çıkmıştır (Tablo III).

TABLO III
Proje Tanımındaki Örneğin Çıktısı

Komut	Olay	Konum	Batarya	Teslim
ILERI 2	HAREKET	(3, 1)	18	0
AL	AL	(3, 1)	18	0
ILERI 5	CARPISMA	(5, 1)	11	0
BIRAK	TESLIM	(5, 1)	11	1

TABLO IV
Örnek Test Paketi Sonuçları

Test	Kapsam	Sonuç
01	Düz hareket	Geçti
02	Duvara çarpma	Geçti
03	Al, taşı, teslim et	Geçti
04	Şarj istasyonu	Geçti
05	Hatalı komut satırları	Geçti
06	R olmayan harita	Geçti (ValueError)

B. Deneme Veri Setleri

Beş deneme veri setinin sonuçları Tablo V'te özetlenmiştir. Tüm setlerde kutuların tamamı teslim edilmiş ve robot son ILERI komutuyla bir kez duvara çarpmıştır. Her setteki kasıtlı hatalı satırlar (ör. AL 1, ILERI −1, küçük harfli sol, SARJ ET, GERI 1, DUR) doğru satır numaralarıyla tespit edilmiş ve simülasyon kesintisiz devam etmiştir.

TABLO V
Deneme Veri Setleri Sonuçları

Set	Harita	Hatalı satır	Teslim	Adım	Batarya	En çok ziyaret
1	16×11	3	4	63	2/163	(6,6) ×5
2	15×10	4	5	70	2/112	(6,4) ×4
3	13×10	3	4	67	94/204	(10,4) ×5
4	12×8	5	5	75	60/148	(1,5) ×6
5	15×11	5	3	34	86/145	(6,3) ×4

Tablo VI, çalıştırılan komutların olay türlerine göre dağılımını gösterir. Tüm setlerde dönüşler (DONUS) hareketlerden fazladır; bu, ızgara haritalarda kısa düz parçalar arasında sık yön değiştirmenin gerektiğini ve dönüş maliyetinin toplam tüketimde önemli bir pay tutabileceğini göstermektedir. GECERSIZ olaylarının tamamı, robot şarj istasyonunda değilken verilen SARJ komutlarından kaynaklanmıştır; bu komutlar kurala uygun olarak bataryayı değiştirmemiştir.

TABLO VI
Çalıştırılan Komutların Olay Dağılımı

Set	HAREKET	DONUS	AL	TESLIM	SARJ	GECERSIZ
1	27	31	4	4	1	1
2	30	37	5	5	1	2
3	29	36	4	4	1	2
4	32	36	5	5	1	2
5	13	17	3	3	1	2

Üretilen log.csv dosyasının veri_1 için ilk satırları Tablo VII'de verilmiştir. Her satır komut çalıştıktan sonraki durumu gösterir; örneğin ilk ILERI 3 komutu 3 × 3 = 9 birim harcayarak bataryayı 163'ten 154'e düşürmüştür.

TABLO VII
veri_1 İçin log.csv Örneği

adim	komut	x	y	yon	batarya	olay
1	ILERI 3	8	4	K	154	HAREKET
2	SOL	8	4	B	153	DONUS
3	ILERI 4	4	4	B	141	HAREKET
4	SAG	4	4	K	140	DONUS
5	ILERI 1	4	3	K	137	HAREKET
6	SOL	4	3	B	136	DONUS
7	ILERI 1	3	3	B	133	HAREKET
8	AL	3	3	B	133	AL

C. Batarya Kısıtının Etkisi

Batarya bitme kuralını sınamak için veri_1 farklı kapasitelerle çalıştırılmıştır (Tablo VIII). Kapasite 163'ten 120'ye düşürüldüğünde robot 69 komuttan yalnızca 28'ini çalıştırabilmiş, bir kutu teslim etmiş ve simülasyon batarya bitti bayrağıyla durmuştur. Kapasite 80 olduğunda ise robot şarj istasyonuna ulaşamadan durmuş ve hiç teslim yapamamıştır. Bu sonuç, durduktan sonraki komutların çalıştırılmadığını da doğrulamaktadır.

TABLO VIII
veri_1 İçin Batarya Kapasitesinin Etkisi

Kapasite	Çalışan komut	Teslim	Kalan	Bitti mi
163	69 / 69	4	2	Hayır
120	28 / 69	1	2	Evet
80	17 / 69	0	3	Evet

D. GIT Komutu ve Sağlamlık

GIT komutu veri_1 haritasında (8, 7) başlangıcından sınanmıştır. (6, 8) teslim noktasına 3 adımda, (13, 8) teslim noktasına 6 adımda ve (7, 5) şarj istasyonuna 3 adımda ulaşılmış; bu değerler el ile sayılan en kısa yollarla aynıdır. Duvar olan (10, 7) hedefi için olay YOL_YOK olmuş ve robot yerinde kalmıştır. Sağlamlık için rastgele boyut, duvar oranı, ayar ve komut dizileriyle (ILERI n için $n \leq 10^6$ dahil) 3000 senaryo çalıştırılmıştır; hiçbirinde hata, sonsuz döngü veya negatif batarya görülmemiş, toplam süre yaklaşık 0,5 saniye olmuştur. Deneme veri setlerinin her biri 1 ms'nin altında çalışmaktadır.

IV. Sonuç

Bu projede dosyalardan okunan bir haritada robot komutlarını belirlenen kurallara göre çalıştıran, sonuçları JSON ve CSV olarak kaydeden ve simülasyonu adım adım gösteren bir program geliştirilmiştir. Kural mantığının arayüzden ayrılması, aynı fonksiyonların hem test betiği hem de arayüz tarafından kullanılmasını sağlamıştır. Tek bir durum sözlüğü ve küçük yardımcı fonksiyonlar (_harca, _don, _bir_adim) kuralların tek bir yerde tanımlanmasına ve kolayca değiştirilebilmesine olanak vermiştir. Tüm örnek testler geçilmiş ve hatalı girdilerde program çökmemiştir.

Çalışmanın bir sınırlılığı, GIT komutunun adım sayısını en aza indirmesi fakat dönüş maliyetini yol seçiminde hesaba katmamasıdır. Konum ve yönü birlikte durum olarak kabul eden ağırlıklı bir arama ile toplam batarya tüketimi en aza indirilebilir. Ayrıca robotun gerektiğinde kendiliğinden şarj istasyonuna gitmesi ve birden fazla robotun çakışmadan çalışması gelecek çalışmalar olarak önerilmektedir. Ekip çalışması sırasında kod Git ile sürüm kontrolünde tutulmuştur [7].

KAYNAKÇA

- 1] Depo robotlarıyla ilgili IEEE makalesi
- [2] *Pro Git* kita